Jules looking for the lost treasure

Bron Matthieu

2022-2023

I) Introduction au projet Zuul sur l'île **4**
II) Réponses aux exercices : **6**
Exercice 7.5 : 6
Exercice 7.6 : 6
Exercice 7.7 : 7
Exercice 7.8 : 7
Exercice 7.8.1 : 8
Exercice 7.9 : 8
Exercice 7.11 : 9
Exercice 7.14 : 9
Exercice 7.15 : 10
Exercice 7.16 : 10
Exercice 7.17 : 11
Exercice 7.18 : 11
Exercice 7.18.1 : 12
Exercice 7.18.3 : 12
Exercice 7.18.4 : 12
Exercice 7.18.6 : 12
Exercice 7.18.8 : 16
Exercice 7.19.2 : 18
Exercice 7.20 : 18
Exercice 7.21 : 19
Exercice 7.22 : 20
Exercice 7.22.2 : 20
Exercice 7.23 : 21
Exercice 7.26 : 22
Exercice 7.28.1 : 22
Exercice 7.29 : 23
Exercice 7.30 : 23
Exercice 7.31 : 25
Exercice 7.31.1 : 26
Exercice 7.32 : 26
Exercice 7.33 : 28
Exercice 7.34 : 29
Exercice 7.42 : 30
Exercice 7.42.2 : 32
Exercice 7.43 : 32
Exercice 7.44 : 33
Exercice 7.46 : 35
Exercice 7.46.1 : 37
Exercices pour le scénario : **39**
Exercice 1 : 39
Exercice 2 : 40
Exercice 3 : 41
Exercice 4 : 41
III) Mode d'emploi (si nécessaire, instructions d'installation ou pour démarrer le jeu) **42**
IV) Déclaration obligatoire anti-plagiat **42**

I) Introduction au projet Zuul sur l'île

A) Auteur

BRON Matthieu, Groupe 2

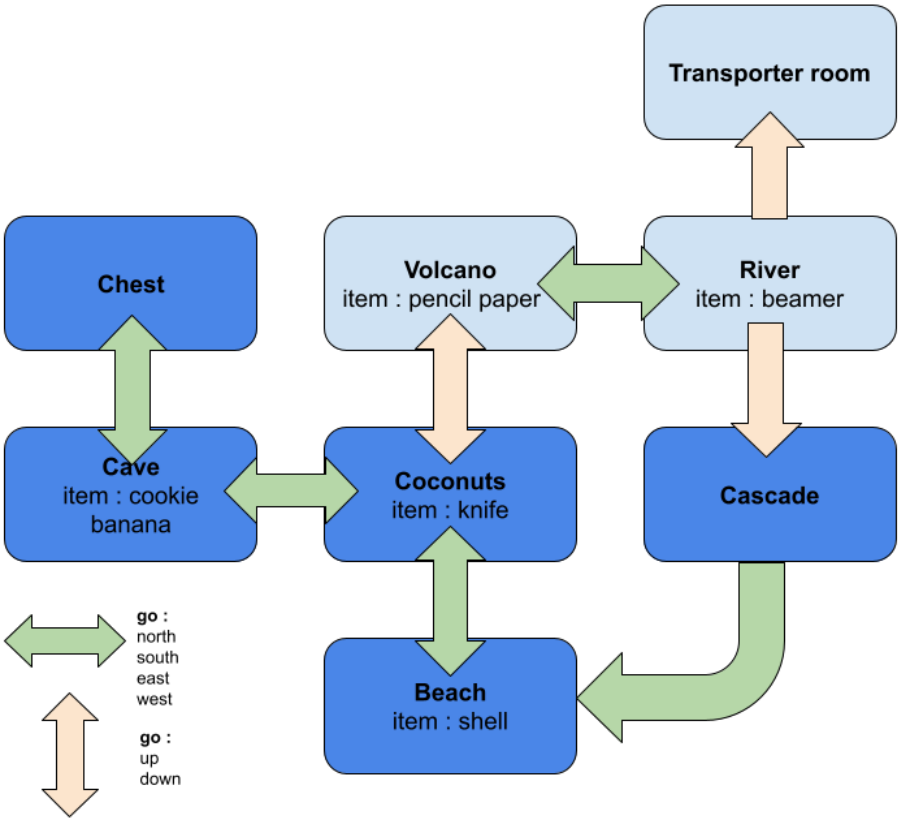
B) Phrase thème :

Le pirate, Jules la Tempête, doit retrouver le trésor indiqué sur une mystérieuse carte

C) Résumé du scénario :

Le pirate commence son aventure sur la plage de l'île. Il est à la recherche d'un trésor perdu sur une île. Pour accéder au trésor, il le sait, il doit passer différentes épreuves qu'il cherche dans différents lieux remarquables de l'île. A l'issue de ces épreuves, il aura deux clés qui lui permettront d'ouvrir le coffre au trésor.

D) Plan de l'île :



E) Scénario détaillé :

Jules a trouvé une carte au trésor dans le grenier de son grand-père. Elle montre une île déserte au milieu de l'océan indien. Plusieurs phrases étranges sont inscrites sur cette carte. On y voit aussi des lieux qu'il faudra sans doute visiter pour résoudre les énigmes et accéder au coffre.

Jules arrive sur la plage de l'île. Il n'y a rien à cet endroit.

Jules doit aller dans la grotte pour récupérer des bananes et les donner aux signes pour obtenir la première clé.

Avec cette clé, Jules peut ouvrir la salle du coffre qui se trouve à côté de la grotte.

F) Détail des lieux, items, personnages :

Beach : c’est le premier lieu du jeu, Jules apparaît seul sur cette plage déserte.
Coconuts : au milieu des cocotiers, il y des singes amateurs de bananes.
Cave : C’est une grotte où il y a des bananes
Chest : c’est la salle du coffre, elle peut être ouverte uniquement si le pirate a obtenu les 2 clés nécessaires.
Volcano : le volcan de l'île, il y a un parchemin avec un texte chiffré.
River : la rivière qui mène à la cascade
Cascade : la cascade
Transporter room : salle permettant de se téléporter vers une autre lorsqu’on y sort

G) Situations gagnantes ou perdantes :

gagnante : accéder à la salle du coffre
perdante : faire plus de 10 déplacements

II) Réponses aux exercices :

Exercice 7.5 :

Pour éviter la duplication de code et afficher la description de la pièce courante ainsi que les sorties disponibles on crée la procédure printLocationInfo() (le code à déjà été réalisé dans goRoom() il suffit de l'insérer dans la nouvelle procédure). On peut alors appeler printLocationInfo() dans printWelcome() et dans goRoom().
La procédure printLocationInfo() évite donc la duplication de code.

```
private void printLocationInfo(){
	System.out.println("You are " + this.aCurrentRoom.getDescription());
	System.out.print("Exits : ");
	if (this.aCurrentRoom.aNorthExit != null){
		System.out.print("north ");
	}
	if (this.aCurrentRoom.aSouthExit != null){
		System.out.print("south ");
	}
	if (this.aCurrentRoom.aEastExit != null){
		System.out.print("east ");
	}
	if (this.aCurrentRoom.aWestExit != null){
		System.out.print("west ");
	}
}
```

Exercice 7.6 :

On écrit un getter getExit() dans la classe Room, pour obtenir l'objet Room qui se trouve dans la direction indiquée en paramètre à partir du lieu actuel.. Cette méthode retourne null si la direction indiquée n'est pas un point cardinal ou s' il n'existe pas de lieu dans la direction indiquée (l'objet Room est initié à null).

```
public Room getExit(final String pDirection){
	if (pDirection.equals("north")){
		return aNorthExit;
	}
	if (pDirection.equals("south")){
		return aSouthExit;
	}
	if (pDirection.equals("east")){
		return aEastExit;
	}
```

```
        if (pDirection.equals("west")){
                return aWestExit;
        }
        return null;
}
```

```
Room vNextRoom = this.aCurrentRoom.getExit(vDirection);
```

Exercice 7.7 :

Dans la classe Room, on crée la méthode getExitsString(), qui permet d'afficher les directions possibles pour sortir d'une salle. On peut alors se servir de cette méthode dans la classe Game pour simplifier la méthode printLocationInfo().

```
public String getExitsString(){
        String vMessageSortie = "Exits : ";
        if(this.getExit("north") != null){
                vMessageSortie = vMessageSortie + "north";
        }
        if(this.getExit("south") != null){
                vMessageSortie = vMessageSortie + "south";
        }
        if(this.getExit("east") != null){
                vMessageSortie = vMessageSortie + "east";
        }
        if(this.getExit("west") != null){
                vMessageSortie = vMessageSortie + "west";
        }
        return vMessageSortie;
}
```

```
private void printLocationInfo(){
        System.out.println("You are " + this.aCurrentRoom.getDescription());
        System.out.println(this.aCurrentRoom.getExitString());
}
```

Exercice 7.8 :

Tout d'abord, on crée une hashMap exits qui permettra de stocker les Rooms aNorthExit, aSouthExit, aEastExit et aWestExit en les associant avec leurs directions respectives north, south, east et west. Cela permettra d'accéder directement à l'objet Room à partir de sa direction. On change alors le constructeur naturel de la classe Room et le getter getExit() qui doit également être modifié pour rechercher dans une hashMap. Nous remplaçons à présent la

méthode setExits() par une autre méthode setExit() qui permet de positionner les différentes Room par rapport à un lieu et une direction.. Il devient nécessaire de changer la méthode createRooms() de la classe Game pour prendre en compte ces modifications. Voici un exemple pour la salle Coconuts.

```
import java.util.HashMap;
public class Room
{
	private HashMap<String, Room> exits;
} //déclaration de la HashMap
```

```
public Room (final String pDescription){
	this.aDescription = pDescription;
	this.exits = new HashMap<String, Room>();
}
```

```
public Room getExit(final String pDirection){
	return exits.get(pDirection);
}
```

```
public void setExit(final String pDirection, final Room pNeighbor){
	exits.put(pDirection, pNeighbor);
}
```

```
//sorties Conuts
	vCoconuts.setExit("north", vVolcano);
	vCoconuts.setExit("south", vBeach);
	vCoconuts.setExit("west", vCave);
```

Exercice 7.8.1 :

Les salles Volcano et River sont maintenant en hauteur, on peut monter ou descendre au niveau du volcan mais il est seulement possible de descendre de la rivière.

Exercice 7.9 :

Suite à la création de la hashMap exits, il faut réécrire la méthode getExitString() car elle ne permet pas de manipuler ce type de variable. Nous utilisons la méthode keySets() d'un objet hashMap pour obtenir l'ensemble des sorties possibles. Pour cela, il faut importer la classe Set au niveau de la classe Room. Cela nous permettra de manipuler le Set de String vEnsembleCles contenant la liste des sorties d'un lieu. Finalement, on obtient :

```
public String getExitString(){
	String vMessageSortie = "Exits :";
	Set<String> vEnsembleCles = aExits.keySet();
	//vEnsembleCles sont les directions possibles "north", "south" "up" etc.
	for ( String vNbExits : vEnsembleCles){
		vMessageSortie = vMessageSortie + vEnsembleCles;
	}
	return vMessageSortie;
}
```

Exercice 7.11 :

Pour réduire le couplage entre la classe Game et la classe Room, il faut créer la méthode getLongDescription() dans la classe Room. Cela permet d'appeler cette méthode depuis la méthode printLocationInfo() de la classe Game.

```
public String getLongDescription(){
	return "You are " + aDescription + '\n' + getExitString();
}
```

```
private void printLocationInfo(){
	System.out.println(aCurrentRoom.getLongDescription());
}//printLocationInfo
```

Exercice 7.14 :

Si on souhaite ajouter de nouvelles commandes en plus que celles existantes, il est préférable d'initialiser un tableau de String aValidCommands[ ] dans la classe CommandWords. Cette liste contiendra l'ensemble des commandes valides. Après avoir ajouté une commande dans le tableau aValidCommands(), il faut créer une méthode dans la classe Game pour qu'elle soit associée à la commande au travers la méthode processCommand(). Ici, la commande "look" appelle la méthode look() de la classe Game.

```
private static final String aValidCommands[] = {"go", "help", "quit", "look"};
```

```
private boolean processCommand(final Command pCommandProcess){
	//if(...)
	...
	else if (pCommandProcess.getCommandWord().equals("look")){
		look();
		return false;
```

```
        }
}//processCommand
```

```
private void look(){
        System.out.println(aCurrentRoom.getLongDescription());
        return;
}
```

Exercice 7.15 :

De la même manière que nous avons ajouté la commande “look” on ajoute la commande “eat”. On obtient alors un nouveau tableau de commandes valides. Le code ci-dessous illustre la création de la commande eat().

```
private static final String aValidCommands[] = {"go", "help", "quit", "look",
"eat"};
```

```
private void eat(){
        System.out.println("You have eaten now and you are not hungry any more");
        return;
        }
```

```
private boolean processCommand(final Command pCommandProcess){
        //if(...)
        ...
        else if (pCommandProcess.getCommandWord().equals("eat")){
                eat();
                return false;
        }
}//processCommand
```

Exercice 7.16 :

Après avoir ajouté les commandes “look” et “eat”, la méthode help() ne fonctionne plus car il est possible d’utiliser de nouvelles commandes. Pour corriger cela, on crée la méthode showAll(), dans la classe CommandWords. Ensuite, pour éviter de nouveaux liens entre les classes, on crée la méthode showCommands() dans la classe Parser. Ce qui permettra de faire le relai entre les classes Game et CommandWords. Il faut alors modifier la méthode printHelp() de la classe Game en utilisant ce qui vient d’être fait.

```
public void showAll(){
```

```
	for(String vCommand : aValidCommands){
		System.out.print(vCommand + ' ');
	}
	System.out.println();
}
```

```
public void showCommands(){
	this.aValidCommands.showAll();
}
```

```
private void printHelp(){
	System.out.println("Comme on, try to find this treasure");
	System.out.print("Your command words are : ");
	aParser.showCommands();
}
```

Exercice 7.17 :

Si l'on souhaite ajouter une nouvelle commande, il faudra toujours modifier la classe Game à cause de la méthode processCommand() qui permet de reconnaître les commandes/mots clés pour faire certaines actions.

Exercice 7.18 :

On ne veut plus que la classe CommandWords affiche directement les sorties possibles (autrement dit, on veut limiter le couplage implicite entre les classes). On va alors créer une méthode getCommandList(). Elle fonctionne de la même manière que showAll() en affichant l'ensemble des commandes du tableau **aValidCommand**. Ensuite il faut modifier la méthode showCommands() de la classe Parser. Finalement, on modifie la méthode printHelp() de la classe Game pour afficher en console les différentes commandes valides.

```
public String getCommandList(){
	String vChaineDesCommandes = "";
	for(String vCommand : aValidCommands){
		vChaineDesCommandes = vChaineDesCommandes + vCommand + " ";
	}
	return vChaineDesCommandes;
}
```

```
public String showCommands(){
	return this.aValidCommands.getCommandList();
```

```
}
```

```
private void printHelp(){
	System.out.println("Comme on, try to find this treasure");
	System.out.print("Your command words are : ");
	System.out.println(aParser.showCommands());
}//printHelp
```

Exercice 7.18.1 :

Comparaison avec le projet zuul-better, aucune modification n'a été réalisée sur mon projet.

Exercice 7.18.3 :

Le choix des photos à été effectué.

Exercice 7.18.4 :

Le titre du jeu est : Jules looking for the lost treasure
le message de bienvenue est affiché à l'aide de la méthode printWelcome() de la classe Game

Exercice 7.18.6 :

On souhaite ajouter des images à notre jeu.

- Les classes Command et CommandWords ne changent pas
- Dans la classe Room :
  - On ajoute un attribut et on modifie le constructeur de cette classe. Il faut alors modifier la méthode getExitsString() et on crée une nouvelle méthode getImageName() qui retourne le nom de l'image.
- Dans la classe Parser :
  - On n'utilise plus la classe Scanner mais StringTokenizer. On retire alors l'attribut qui permettait de lire les commandes en entrées au clavier et on ajoute l'attribut **aCommandWords**. Le constructeur change légèrement. Finalement, on change la méthode getCommand().
- Dans la classe Game :
  - On transfère tout le code de la classe Game (sauf la méthode quit() qui sera remplacée par une méthode endGame() dans une nouvelle classe GameEngine.
- Dans la classe GameEngine :
  - On y incorpore le code de l'ancienne classe Game. On ajoute une méthode setGUI() qui permet de modifier l'interface du jeu. On peut alors remplacer les System.out.print() par aGui.print() afin d'afficher les messages dans l'interface graphique et non dans un terminal. La procédure printWelcome() change aussi. Dans la méthode createRooms(), chaque lieu a maintenant une description et

une image associée. Il faut nommer ces images. On ne peut plus utiliser le nom de méthode processCommand() puisqu'elle est utilisée dans la classe UserInterface. On va donc renommer cette méthode en interpretCommand() dans la classe GameEngine. Celle-ci permet d'analyser et d'interpréter ce qui est tapé au clavier par le joueur. Il faut écrire la méthode endGame() pour arrêter le jeu. Il faut également modifier la méthode goRoom() pour afficher à l'écran le lieu suivant. Finalement il faut modifier la méthode printLocationInfo() pour afficher la description du lieu et l'image associée.

- Dans la classe UserInterface :
  - Pour ne pas importer de classes inutiles dans UserInterface, il est préférable de n'importer que les classes qui sont nécessaires.

```
private String aImageName;
```

```
public Room (final String pDescription, final String pImage){
	this.aDescription = pDescription;
	this.aExits = new HashMap<String, Room>();
	this.aImageName = pImage;
}//Room
```

```
public String getExitString(){
	StringBuilder returnString = new StringBuilder( "Exits:" );
	Set<String> vEnsembleCles = aExits.keySet();
	//vEnsembleCles sont les directions possibles "north", "south" "up" etc
	for ( String vNbExits : vEnsembleCles){
		returnString.append(" " + vNbExits);
	}
	return returnString.toString();
}//getExitString
```

```
public String getImageName()
{
	return this.aImageName;
}
```

```
private CommandWords aValidCommands;
private CommandWords aCommandWords;
```

```
public Parser(){
	this.aValidCommands = new CommandWords();
	this.aCommandWords = new CommandWords();
} // Parser()
```

```java
public Command getCommand( final String pInputLine ){
	String vWord1;
	String vWord2;

	StringTokenizer tokenizer = new StringTokenizer( pInputLine );

	if ( tokenizer.hasMoreTokens() )
		vWord1 = tokenizer.nextToken();		// get first word
	else
		vWord1 = null;

	if ( tokenizer.hasMoreTokens() )
		vWord2 = tokenizer.nextToken();		// get second word
	else
		vWord2 = null;

	// Now check whether this word is known. If so, create a command
	// with it. If not, create a "null" command (for unknown command).

	if ( this.aCommandWords.isCommand( vWord1 ) )
		return new Command( vWord1, vWord2 );
	else
		return new Command( null, vWord2 );
} // getCommand()
```

```java
public class Game{

    private UserInterface aGui;
    private GameEngine aEngine;

	public Game()
	{
	this.aEngine = new GameEngine();
	this.aGui = new UserInterface( this.aEngine );
	this.aEngine.setGUI( this.aGui );
	}
}
```

```java
public void setGUI( final UserInterface pUserInterface ){
	this.aGui = User Interface;
	this.printWelcome();
}
```

```java
private void printWelcome(){
	this.aGui.println("Welcome to the World of The Island !");
	this.aGui.println("Jules is looking for the lost treasure of the Island");
	this.aGui.println("Now, you have to help Jules to find this treasure");
	this.aGui.println("Type 'help' if you need help");
	this.aGui.println();
	printLocationInfo();
	this.aGui.showImage(this.aCurrentRoom.getImageName());
}//printWelcome
```

```java
private void createRooms (){
	// déclaration des 7 lieux
	Room vBeach = new Room("on the beach of the desert island","beach.jpg");
	Room vCoconuts = new Room("at the coconut palms", "coconuts_palms.jpg");
	Room vCave = new Room("in the dark cave", "cave.jpg");
	Room vVolcano = new Room("on top of the volcano", "top_volcano.jpg");
	Room vRiver = new Room("Up the river", "river.jpg");
	Room vCascade = new Room("fall down the cascade", "cascade.jpg");
	Room vChest = new Room("the room of the treasure","close_treasure.jpg");

	//...

}//createRooms
```

```java
public void interpretCommand( final String pCommandLine )
	{
	this.aGui.println( "> " + pCommandLine );
	Command vCommand = this.aParser.getCommand( pCommandLine );

	if ( vCommand.isUnknown() ) {
		this.aGui.println( "I don't know what you mean..." );
		return;
	}

	String vCommandWord = vCommand.getCommandWord();
	if ( vCommandWord.equals( "help" ) )
		this.printHelp();
	else if ( vCommandWord.equals( "go" ) )
		this.goRoom( vCommand );
	else if (vCommandWord.equals("look"))
		look();
	else if (vCommandWord.equals("eat"))
		eat();
	else if ( vCommandWord.equals( "quit" ) ) {
		if ( vCommand.hasSecondWord() )
```

```
            this.aGui.println( "Quit what?" );
            else
            this.endGame();
        }
        else {
            aGui.println("Unknown command");
        }
}
```

```
private void endGame(){
        this.aGui.println( "Thank you for playing.  Good bye." );
        this.aGui.enable( false );
}
```

```
private void printLocationInfo(){
        aGui.println(this.aCurrentRoom.getLongDescription());
        aGui.println(this.aCurrentRoom.getImageName());
}//printLocationInfo
```

```
//import javax.swing.*;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JLabel;
import javax.swing.ImageIcon;
import javax.swing.JScrollPane;
import javax.swing.JPanel;

//import java.awt.*;
import java.awt.Dimension;
import java.awt.BorderLayout;

//import java.awt.event.*;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

import java.net.URL;

//import java.awt.image.*;
import java.awt.event.ActionEvent;
```

Exercice 7.18.8 :

Pour créer un bouton, il faut importer la classe `javax.swing.JButton;` dans la classe UserInterface. On crée alors un attribut pour chaque bouton. Il faut alors modifier le constructeur de la classe UserInterface pour initialiser les boutons. Ensuite, on crée un nouveau panel de boutons dans la méthode createGUI() avec `JPanel vPanelButtons = new JPanel();` On crée alors les 4 boutons dans la méthode createGUI(). Finalement, on place le panel des boutons à l'est de la fenêtre. Nous avons maintenant nos 4 boutons mais aucune action ne leur a été assignée. On ajoute donc un listener pour chaque bouton. Enfin on associe le clic du bouton à une action dans la méthode actionPerformed(). J'ai par la suite ajouté des boutons up et down pour le déplacement entre certains lieux. Ces deux boutons sont créés dans un panel. Nous avons donc au final un panel de direction qui contient 2 panels (un pour les 4 boutons de direction et un pour les 2 boutons up et down).

```
private JButton     aButtonNorth;
private JButton     aButtonSouth;
private JButton     aButtonEast;
private JButton     aButtonWest;
```

```
public UserInterface( final GameEngine pGameEngine )
       {
       this.aEngine = pGameEngine;
       this.createGUI();
       this.aEngine.setGUI(this);
} // UserInterface(.)
```

```
this.aButtonNorth = new JButton("Go north!");
this.aButtonSouth = new JButton("Go south!");
this.aButtonEast = new JButton("Go east!");
this.aButtonWest = new JButton("Go west!");
```

```
vPanel.add(vPanelButtons, BorderLayout.EAST);
```

```
this.aButtonNorth.addActionListener(this);
this.aButtonSouth.addActionListener(this);
this.aButtonEast.addActionListener(this);
this.aButtonWest.addActionListener(this);
```

```
public void actionPerformed( final ActionEvent pE )
       {
       if(this.aButtonNorth.equals(pE.getSource())){
              this.processCommand("go north");
       }
```

```
        else if(this.aButtonSouth.equals(pE.getSource())){
                this.processCommand("go south");
        }
        else if(this.aButtonEast.equals(pE.getSource())){
                this.processCommand("go east");
        }
        else if(this.aButtonWest.equals(pE.getSource())){
                this.processCommand("go west");
        }
        else if(this.aButtonUp.equals(pE.getSource())){
                this.processCommand("go up");
        }
        else if(this.aButtonDown.equals(pE.getSource())){
                this.processCommand("go down");
        }
        else {
                // no need to check the type of action at the moment
                // because there is only one possible action (text input) :
                this.processCommand(this.aEntryField.getText());
        }
} // actionPerformed(.)
```

Exercice 7.19.2 :

On déplace l'ensemble des images dans le dossier Images. On doit alors modifier la méthode createRooms() de la classe GameEngine :

```
Room vCave = new Room("in the dark cave", "Images/cave.jpg");
```

(Exemple pour une seule pièce)

Exercice 7.20 :

On souhaite ajouter des items à notre jeu. Pour cela, on crée une classe Items, qui gère les objets Items qui se trouvent dans un lieu. Il faut alors ajouter un nouvel attribut **aItem** à la classe Room. On écrit un setter setItem() qui sera appelé pour associer un Item à un lieu. Puis un getter getItem() utilisé pour obtenir l'item présent dans un lieu. Pour définir les différents items que l'on souhaite mettre dans les différents lieux du jeu, il faut les ajouter dans la classe GameEngine de la même manière qu'on définit les différentes sorties. Pour afficher l'item présent dans un lieu, il faut modifier la méthode getLongDescription() de la classe Room. De cette façon, il n'est pas nécessaire de modifier la procédure printLocationInfo() de la classe GameEngine.

```
public class Item
{
    private String aDescription;
```

```
    private int aPoids;
    //...
}
```

```
private Item aItem;
```

```
public void setItem(final Item pItem){
      this.aItem = pItem;
}
```

```
public Item getItem(){
      return this.aItem;
}
```

```
//Item dans le lieu Conuts
vCoconuts.setItem(new Item("some bananas", 4));
```

```
public String getLongDescription(){
      String vDescription = "You are " + aDescription + '\n' + getExitString() +
'\n' ;
      if(this.aItem != null){
            vDescription = vDescription + "There is " +
this.aItem.getDescription() + '\n' + "Item weight : " + this.aItem.getPoids();
      }
      else {
            vDescription = vDescription + "there is no Items here";
      }
      return vDescription;
}
```

Exercice 7.21 :

Les informations concernant l'item doivent se trouver dans la classe Item. Il faut créer la méthode getItemDescription() dans la classe Item. Il faut également modifier la méthode getLongDescription() de la classe Room pour afficher les informations liées à l'item.

```
 public String getItemDescription(){
      return "there is " + this.aDescription + "Item weight : " + this.aPoids;
}
```

```
public String getLongDescription(){
	String vDescription = "You are " + aDescription + '\n' + getExitString() +
'\n' ;
	vDescription = vDescription + aItem.getItemDescription();
	return vDescription;
}
```

Exercice 7.22 :

Une solution pour mettre plusieurs items dans une même pièce est de les stocker dans une hashmap. Pour cela on ajoute l'attribut **aItems** dans la classe Room. Il faut alors l'initialiser dans le constructeur (même instruction) et modifier la méthode getItem() pour obtenir un Item à partir de sa clé (son nom). Ensuite on remplace la procédure setItem() par addItem(). Celle-ci permet d'ajouter plusieurs Items à un lieu. Finalement, on crée une méthode getItemString() qui permettra d'afficher le nom de l'item sur l'interface.

```
private HashMap<String, Item> aItems;
```

```
public Item getItem(final String pNom){
	return this.aItems.get(pNom);
}
```

```
public void addItem(final String pNom, final Item pItem){
	this.aItems.put(pNom, pItem);
}
```

```
public String getItemString (){
	StringBuilder returnString = new StringBuilder( "Items:" );
	Set<String> vEnsembleCles = aItems.keySet();
	//vEnsembleCles sont les noms des objets dans la piece
	for ( String vCle : vEnsembleCles){
		returnString.append(" " + vCle);
	}
	return returnString.toString();
}
```

Exercice 7.22.2 :

On peut alors ajouter différents items dans une pièce voici un exemple d'ajout d'item

```
vVolcano.addItem("pencil", new Item("a pencil", 3));
```

```
vVolcano.addItem("paper", new Item("a piece of paper", 2));
```

Exercice 7.23 :

Pour ajouter une nouvelle commande "back" on commence par l'ajouter à la liste de commandes valides de la classe CommandWords. Dans cet exercice, je ne commente pas la première version où l'action back ne fonctionnait qu'une seule fois. J'ai pris la décision d'utiliser une **LinkedList** comme vu lors du dernier amphi. Ainsi, on crée un nouvel attribut aPreviousRoom dans la classe GameEngine. On initialise la liste à null dans la méthode createRooms(). Cela permet de ne pas pouvoir revenir en arrière au démarrage du jeu. Il faut remplir cette liste depuis la procédure goRoom(), l'information sur le lieu dans lequel le joueur se situe est stockée dans l'attribut **aCurrentRoom** et le lieu d'où il vient est lui dans l'attribut **aPreviousRoom**. On crée la procédure back() qui remonte la liste en affectant le dernier lieu de **aPreviousRoom** à **aCurrentRoom** puis en l'effaçant de la liste. Finalement, pour que la commande back soit reconnue, on l'ajoute à la liste des commandes valides et on crée une condition dans la procédure interpretCommand().

```
private static final String aValidCommands[] = {"go", "back", "help", "quit",
"look", "eat", "back"};
```

```
private LinkedList<Room> aPreviousRoom = new LinkedList<Room>();
```

```
private void createRooms () {

	this.aPreviousRoom.add(null);
}//createRooms
```

```
private void goRoom (final Command pCommand){
	...
	this.aPreviousRoom.addLast(this.aCurrentRoom);
}// goRoom
```

```
private void back(){
	// test au début car aPreviousRoom vaut null
	if(this.aPreviousRoom.getLast() == null){
		this.aGui.println("Can't go back!!!");
		return;
	}
	this.aCurrentRoom = this.aPreviousRoom.getLast();
	this.aPreviousRoom.removeLast();
	this.aGui.println( this.aCurrentRoom.getLongDescription() );
```

```
        if ( this.aCurrentRoom.getImageName() != null )
              this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

```
else if ( vCommandWord.equals( "back" ) )
            back();
```

Exercice 7.26 :

J'ai résolu le problème des aller-retours entre deux lieux lorsque l'on appuyait plusieurs fois sur back avec une LinkedList lors de l'exercice 7.23.

Exercice 7.28.1 :

On souhaite automatiser les tests, au lieu de taper beaucoup de commandes, on les place dans un fichier texte pour gagner du temps et éviter d'éventuelles erreurs. On commence par importer les classes qui nous seront utiles. Il faut ajouter une nouvelle commande "test" à la liste de commandes valides de la classe CommandWords. Ensuite, pour que la commande test soit reconnue, on ajoute une condition dans la procédure interpretCommand(). Finalement, on crée la procédure test() qui permet de lire un fichier texte (.txt) et d'exécuter ligne par ligne ce qui est écrit dedans. Chaque ligne est considérée comme une commande saisie par l'utilisateur. De cette manière, il est possible d'enchaîner toutes les lignes du fichier texte comme autant de commandes d'un joueur.

```
import java.util.LinkedList;
import java.io.InputStream;
import java.util.Scanner;
```

```
public void interpretCommand( final String pCommandLine ){
        ...
         else if (vCommandWord.equals("test"))
              test(vCommand);
        ...
}
```

```
private void test(final Command pCommand){
  if(pCommand.hasSecondWord() == false){
       this.aGui.println("Need a file name");
       return;
  }

  String vFileName = pCommand.getSecondWord();
```

```
  vFileName = vFileName + ".txt";
  //lire le fichier
  Scanner vSC;
  InputStream vIS =
this.getClass().getClassLoader().getResourceAsStream(vFileName);
  if(vIS != null){
       vSC = new Scanner( vIS );
       while(vSC.hasNextLine()){
              String vLigne = vSC.nextLine();
              this.aGui.processCommand(vLigne);
       }
  }
}
```

Exercice 7.29 :

L'objectif de cet exercice est de centraliser toutes les informations concernant un joueur dans la classe Player. L'ensemble des informations étaient jusque là contenues dans la classe GameEngine. Il faut alors déplacer une partie de ces informations dans la classe Player. Pour cela, on crée 3 attributs associés à un joueur (son nom, où il se trouve et le lieu où il se trouvait). On crée alors différents getters et setters de ces attributs, qui seront utilisés dans la classe GameEngine.

```
private String aName;
private Room aCurrentRoom;
private LinkedList<Room> aPreviousRoom = new LinkedList<Room>();
```

Exercice 7.30 :

On souhaite prendre et poser un objet. Pour ce faire, on ajoute les mots take et drop à la liste de commandes valides de la classe CommandWords. Dans la classe Player, on ajoute un attribut qui correspond à l'item porté par le joueur. Il faut donc être capable d'ajouter un Item au joueur avec la procédure setItem() que l'on crée dans la classe Player. Dans la classe Room, on ajoute la procédure removeItem() qui permet de retirer un item d'un lieu. Ensuite, dans la classe GameEngine, on ajoute une procédure take() qui prend en paramètre le nom d'un item, vérifie qu'il est présent dans le lieu et l'ajoute à l'attribut **aItem** du joueur. Il est ensuite retiré du lieu dans lequel se trouve le joueur. Pour poser un Item, on commence par créer un getter getItem() dans la classe Player et on ajoute la procédure removeItem(). On crée la méthode drop() dans la classe GameEngine. Elle prend en paramètre le nom d'un item, l'ajoute à la pièce dans laquelle se trouve le joueur et le retire de l'attribut du joueur. Enfin, pour reconnaître les mots "take" et "drop", on ajoute deux conditions dans la méthode interpretCommand() de la classe GameEngine.

```java
private static final String aValidCommands[] = {"go", "back", "help", "quit",
"look", "eat", "test", "take", "drop" };
```

```java
private Item aItem;
```

```java
public void setItem(final Item pItem){
	this.aItem = pItem;
}
```

```java
public void removeItem(final String pName){
	this.aItems.remove(pName);
}
```

```java
private void take (final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Take what?");
		return;
	}

	String vItemName = pCommand.getSecondWord();
	Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);
	if(vItem == null){
		this.aGui.println("I can't find any " + vItemName + "!");
		return;
	}
	this.aPlayer.setItem(vItem);
	this.aPlayer.getCurrentRoom().removeItem(vItemName);
}
```

```java
public Item getItem(){
	return this.aItem;
}
```

```java
public void removeItem(){
	this.aItem = null;
}
```

```java
private void drop (final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Drop what?");
		return;
```

```
        }

        String vItemName = pCommand.getSecondWord();
        Item vItem = this.aPlayer.getItem();
        if(vItem == null){
                this.aGui.println("I can't find any " + vItemName + "!");
                return;
        }
        this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
        this.aPlayer.removeItem();
}
```

```
else if (vCommandWord.equals("take"))
        take(vCommand);
else if (vCommandWord.equals("drop"))
        drop(vCommand);
```

Exercice 7.31 :

Dans cet exercice, l'objectif est de porter plusieurs items. Pour répondre à ce problème, l'attribut aItem de la classe Player est remplacé par l'attribut **aItems** de type HashMap. Le constructeur est modifié pour prendre en compte ce changement de type. Il faut alors modifier les méthodes getItem(), addItem() et removeItem() pour qu'elles manipulent une hashMap d'Items et non plus un unique Item.

```
private HashMap<String, Item> aItems;
```

```
public Player(final String pName){
        this.aName = pName;
        this.aItems = new HashMap<String, Item>();
}
```

```
public Item getItem(final String pNom){
        return this.aItems.get(pNom);
}
```

```
public void addItem(final String pNom, final Item pItem){
        this.aItems.put(pNom, pItem);
}
```

```
public void removeItem(final String pNom){
```

```
        this.aItems.remove(pNom);
}
```

Exercice 7.31.1 :

On crée une nouvelle classe ItemList. Cette classe permet de mutualiser tout ce qui concerne les items (sous forme de hashmap), que ce soit pour les Rooms ou pour les Players. On modifie alors certaines méthodes dans les classes Player et Room pour que tout fonctionne correctement.

```
import java.util.HashMap;
import java.util.Set;

public class ItemList
{
        private HashMap<String, Item> aItems;

        public ItemList(){
        this.aItems = new HashMap<String, Item>();
        }

        public void addItem(final String pNom, final Item pItem){
        this.aItems.put(pNom, pItem);
        }


        public Set<String> getKeys(){
        return this.aItems.keySet();
        }

        public Item getItem(final String pNom){
        return this.aItems.get(pNom);
        }

        public void removeItem(final String pNom){
        this.aItems.remove(pNom);
        }
}
```

Exercice 7.32 :

Pour commencer, on crée deux nouveaux attributs dans la classe Player, le premier **aPoids** pour connaître le poids que porte le joueur et le second **aPoidsMax** pour fixer le poids maximum que le joueur peut porter (**aPoids** est initialisé à 0 dans le constructeur). On crée un getter getPoids() et un setter setPoids() pour l'attribut **aPoids**. Pour pouvoir afficher "too heavy"

si l'objet est trop lourd, il faut modifier les méthodes take() et drop() pour qu'elles prennent en compte le poids des items.

```
private final int aPoidsMax = 8;
private int aPoids;
```

```
public int getPoids(){
	return this.aPoids;
}
```

```
public void setPoids(final int pPoids){
	this.aPoids = pPoids;
}
```

```
private void take (final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Take what?");
		return;
	}

	String vItemName = pCommand.getSecondWord();
	Item vItem = this.aPlayer.getCurrentRoom().getItem(vItemName);
	if(vItem == null){
		this.aGui.println("I can't find any " + vItemName + "!");
		return;
	}
	int vPoids = this.aPlayer.getPoids() + vItem.getPoids();
	if(vPoids <= this.aPlayer.getPoidsMax()){
		this.aPlayer.setPoids(vPoids);
		this.aPlayer.addItem(vItemName, vItem);
		this.aPlayer.getCurrentRoom().removeItem(vItemName);
		this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription()
);
	}
	else {
		this.aGui.println("Too heavy!!!");
	}
}
```

```
private void drop (final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Drop what?");
		return;
```

```
        }

        String vItemName = pCommand.getSecondWord();
        Item vItem = this.aPlayer.getItem(vItemName);
        if(vItem == null){
                this.aGui.println("I can't find any " + vItemName + "!");
                return;
        }
        this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
        this.aPlayer.removeItem(vItemName);
        int vPoids = this.aPlayer.getPoids() - vItem.getPoids();
        this.aPlayer.setPoids(vPoids);
        this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
}
```

Exercice 7.33 :

Pour avoir des informations sur l'inventaire des Items transportés par le joueur, on commence par créer la méthode getItemsKeys() dans la classe Itemlist. Elle permet de retourner l'ensemble des clés d'une hashmap d'items. Autrement dit, les noms des items. Dans la classe Player on crée la méthode getItemsList() pour qu'elle appelle getItemsKeys() et retourne une chaîne de caractères indiquant les items portés par le joueur. Enfin, dans la classe GameEngine, on ajoute la méthode inventory() qui permet d'afficher à l'écran les items portés par le joueur. Pour améliorer le couplage, il est préférable d'utiliser la méthode getItemsKeys() de la classe Itemlist dans la méthode getLongDescription() de la classe Room.

```
public String getItemsKeys(){
        StringBuilder returnString = new StringBuilder( "Items:" );
        Set<String> vEnsembleCles = this.getKeys();
        //vEnsembleCles sont les noms des objets dans la pièce
        for ( String vCle : vEnsembleCles){
                        returnString.append(" " + vCle);
        }
        return returnString.toString();
}
```

```
public String getItemsList(){
         return this.aItems.getItemsKeys();
}
```

```
private void inventory(){
        this.aGui.println(this.aPlayer.getItemsList());
        int vPoids = this.aPlayer.getPoids();
```

```
        String vStr = Integer.toString(vPoids);
        this.aGui.println("Total weight: " + vStr);
        return;
}
```

```
public String getLongDescription(){
        String vDescription = "You are " + aDescription + '\n' + getExitString() +
'\n' ;
        //vDescription = vDescription + getItemString();
        vDescription = vDescription + this.aItems.getItemsKeys();
        return vDescription;
}
```

Exercice 7.34 :

L'objectif de cet exercice est d'ajouter un item "cookie" qui permet d'augmenter le poids maximal que le joueur peut transporter. On commence par modifier l'attribut **aPoidMax** de la classe Player pour qu'il ne soit plus final. Pour manger le cookie, il faut le prendre et donc l'avoir dans l'inventaire du joueur. On reprend la méthode eat() de l'exercice 7.15 et on modifie cette dernière pour qu'elle prenne en paramètre une commande de deux mots. Il faut faire attention car pour que la méthode eat() fonctionne, il faut que l'item soit comestible. Il faut alors que le second mot soit "cookie" (il est possible de mettre directement le mot cookie dans le test car pour le moment c'est le seul item comestible) sinon on affiche qu'il est impossible de manger le nom passé en second mot. Manger le cookie exécute la méthode superPower() qui augmente aPoidsMax à 13 et supprime le cookie de l'inventaire du joueur.

```
private int aPoidsMax = 8;
```

```
private void eat(final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Eat what?");
		return;
	}

	String vItemName = pCommand.getSecondWord();
	Item vItem = this.aPlayer.getItem(vItemName);
	if(vItem == null){
		this.aGui.println("I can't find any " + vItemName + "!");
		return;
	}

	if(!vItemName.equals("cookie")){
		this.aGui.println("I can't eat " + vItemName + "!");
		return;
	}

	this.aPlayer.superPower();
	this.aPlayer.removeItem(vItemName);
	// get poids player
	int vPoidsPlayer = this.aPlayer.getPoids();
	int vPoidsItem = vItem.getPoids();
	this.aPlayer.setPoids(vPoidsPlayer - vPoidsItem);
	this.aGui.println("You have eaten magic cookie now you can carry more
stuff!!");
	return;
}//eat
```

```
public void superPower(){
	this.aPoidsMax = 13;
}
```

Exercice 7.42 :

On commence par créer un nouvel attribut dans la classe Player pour compter le nombre de déplacements. Ensuite on crée une méthode addDeplacement() permettant d'incrémenter cet attribut. Par la suite, il faut créer un getter getNbDeplacements() qui retourne le nombre de déplacements effectués. Dans la classe GameEngine, on commence par créer une méthode lose() qui est appelée quand le joueur perd. Cette méthode affiche à l'utilisateur qu'il a perdu et met fin au jeu. On ajoute un nouvel attribut qui fixe le nombre de déplacements maximum que le joueur peut faire. Il faut alors modifier la méthode goRoom() en ajoutant une condition : si le nombre de déplacements est supérieur à l'attribut aMaxDeplacement, alors il faut appeler la méthode lose(). Sinon, le joueur peut se déplacer et si il se déplace, alors on incrémente l'attribut aNbDeplacements de la classe Player.

```java
private int aNbDeplacements;
```

```java
public void addDeplacement(){
	this.aNbDeplacements++;
}
```

```java
public int getNbDeplacements(){
	return this.aNbDeplacements;
}
```

```java
private void lose(){
	this.aGui.println( "You lose!" );
	this.endGame();
}
```

```java
private final int aMaxDeplacement = 17;
```

```java
private void goRoom (final Command pCommand){
	//ajout de ce if
	if(this.aPlayer.getNbDeplacements() >= this.aMaxDeplacement){
		this.lose();
		return;
	}

	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Go where ?");
		return;
	}

	String vDirection = pCommand.getSecondWord();
	Room vNextRoom = this.aPlayer.getCurrentRoom().getExit(vDirection);

	//pour afficher direction inconnue
	if ( vNextRoom == null )
			this.aGui.println( "There is no door!" );
	//se déplace au nouveau lieu
	else {
		this.aPlayer.setPreviousRoom(this.aPlayer.getCurrentRoom());
		this.aPlayer.setCurrentRoom(vNextRoom);
		this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription()
);
```

```
            if ( this.aPlayer.getCurrentRoom().getImageName() != null )

this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName());
            this.aPlayer.addDeplacement();
        }
}// goRoom
```

Exercice 7.42.2 :

Je fais le choix de conserver l'IHM actuelle car mon interface comporte déjà les boutons de déplacement et les images des lieux en plus de la ligne de commande.

Exercice 7.43 :

Dans cet exercice, l'objectif est de pouvoir utiliser la méthode back() uniquement si il est possible de retourner dans la pièce précédente. Par exemple, dans mon jeu, il est possible de descendre la cascade mais il est impossible de la remonter. On commence par créer une méthode isExit() dans la classe Room qui permet de déterminer si le lieu précédent est une sortie de la pièce dans laquelle se trouve le joueur. Dans la classe GameEngine, il faut ajouter une nouvelle condition à la méthode back() pour s'assurer qu'une pièce est bien une sortie de la pièce dans laquelle le joueur se trouve. Enfin, comme il est impossible de retourner en arrière après avoir franchi la trap door, il est inutile de garder les lieux traversés dans la LinkedList. Pour cela, on crée, dans la classe Player, une méthode resetRoomList() qui permet d'effacer le contenu de la LinkedList. Lors de l'exercice 7.44, je me suis rendu compte que la méthode back ne fonctionnait pas lorsque l'on faisait back plusieurs fois (jusqu'à vider la LinkedList). J'ai donc dû modifier la méthode getPreviousRoom() de la classe Player.

```
public boolean isExit(final Room pRoom){
   Set<String> vEnsembleCles = this.aExits.keySet();
   for ( String vCle : vEnsembleCles){
       Room vRoom = getExit(vCle);
       if(vRoom == pRoom){
             return true;
       }
   }
   return false;
}
```

```
private void back(){
       // test au début car la previous room vaut null
       if(this.aPlayer.getPreviousRoom() == null ||
this.aPlayer.getCurrentRoom().isExit(this.aPlayer.getPreviousRoom()) == false){
             this.aGui.println("Can't go back!!!");
```

```
            this.aPlayer.resetRoomList();
            return;
        }

        this.aPlayer.setCurrentRoom(this.aPlayer.getPreviousRoom());
        this.aPlayer.delPreviousRoom();
        this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
        if ( this.aPlayer.getCurrentRoom().getImageName() != null )
            this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
}//back
```

```
public void resetRoomList(){
        this.aPreviousRoom.clear();
}
```

```
public Room getPreviousRoom(){
        if(this.aPreviousRoom.isEmpty())
            return null;
        else
            return this.aPreviousRoom.getLast();
}
```

Exercice 7.44 :

On souhaite créer un objet beamer qui permet de se téléporter d'un lieu à un autre. Pour utiliser le beamer, faut qu'il soit dans l'inventaire du joueur et le charger dans un lieu, il est utilisable depuis n'importe quel lieu et après utilisation, le beamer est posé (et remis à null) dans le lieu où le joueur est téléporté. Au début du jeu, l'item beamer est au niveau de la rivière. On ajoute les mots "charge" et "fire" à la liste de commandes valides de la classe CommandWords. Dans la classe Player, on ajoute un nouvel attribut qui contient la pièce dans laquelle le beamer est chargé (cet attribut est initialisé à null dans le constructeur). On écrit alors un getter getRoomLoaded() de cet attribut. Ensuite, on crée une méthode setRoomLoaded() permettant d'affecter l'attribut avec le lieu dans lequel le joueur se trouve. Pour que les commandes "charge" et "fire" soient reconnues, on ajoute deux conditions dans la méthode interpretCommand() de la classe GameEngine. Toujours dans la classe GameEngine, on crée la méthode charge(), cette méthode prend une une commande en paramètre. De la même manière, on crée la méthode fire(), qui permet de se téléporter dans la salle où le beamer à été chargé. Après avoir appelé cette méthode, le beamer est remis à null, grâce à la méthode resetRoomLoaded() écrite dans la classe Player. Ensuite, l'objet est posé dans la pièce dans laquelle le joueur est téléporté en utilisant la méthode drop().

```
vRiver.addItem("beamer", new Item("a beamer to go where you want", 6));
```

```
private static final String aValidCommands[] = {"go", "back", "help", "quit",
"look", "eat", "test", "take", "drop", "inventory", "charge", "fire" };
```

```
private Room aRoomLoaded;
```

```
public Room getRoomLoaded(){
	return this.aRoomLoaded;
}
```

```
public void setRoomLoaded(final Room pRoom){
	this.aRoomLoaded = pRoom;
}
```

```
else if (vCommandWord.equals("charge"))
	 charge(vCommand);
else if (vCommandWord.equals("fire"))
	 fire(vCommand);
```

```
private void charge(final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("charge what?");
		return;
	}

	String vItemName = pCommand.getSecondWord();
	Item vItem = this.aPlayer.getItem(vItemName);
	if(vItem == null){
		this.aGui.println("I can't charge any " + vItemName + "!");
		return;
	}

	if(!vItemName.equals("beamer")){
		this.aGui.println("I can't charge " + vItemName + "!");
		return;
	}

	this.aPlayer.setRoomLoaded(this.aPlayer.getCurrentRoom()); //charge le
beamer
	this.aGui.println("Beamer charged!!");
	return;
}//charge
```

```java
public void fire(final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("fire what?");
		return;
	}

	String vItemName = pCommand.getSecondWord();
	Item vItem = this.aPlayer.getItem(vItemName);
	if(vItem == null){
		this.aGui.println("I can't fire any " + vItemName + "!");
		return;
	}

	if(!vItemName.equals("beamer")){
		this.aGui.println("I can't fire " + vItemName + "!");
		return;
	}

	this.aPlayer.setCurrentRoom(this.aPlayer.getRoomLoaded());
	this.aGui.println("Beamer fired!!");

	this.aPlayer.resetRoomLoaded();
	Command vCommand = new Command("drop", vItemName);
	this.drop(vCommand);

	this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
	if ( this.aPlayer.getCurrentRoom().getImageName() != null )
		this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
	this.aPlayer.resetRoomList();
	return;
}
```

```java
public void resetRoomLoaded(){
	this.aRoomLoaded = null;
}
```

Exercice 7.46 :

Cet exercice permet d'ajouter un nouveau lieu : Transporter room. Le joueur peut y entrer comme pour les autres lieux mais il en ressort téléporté dans un autre endroit choisi aléatoirement parmi une liste de lieux possibles. Pour commencer, on crée la classe TransporterRoom qui hérite de la classe Room. Dans la classe GameEngine, et dans la méthode createRooms(), il faut ajouter le lieu Transporter room. Après avoir déclaré ce lieu, on crée un tableau de type Room[ ] contenant les pièces dans lesquelles le joueur peut être téléporté lorsqu'il sort de la Transporter room. Ce tableau est transmis à la classe

TransporterRoom en utilisant la méthode setTargetRooms().. Finalement, on ajoute une sortie en bas de la Transporter room, et une sortie en haut de la rivière. Dans la méthode goRoom() de la classe GameEngine, il faut ajouter une condition permettant de savoir si le joueur se trouve bien dans la Transporter room pour pouvoir appeler la méthode getRandomRoom(). Celle-ci se trouve dans la classe TransporterRoom et permet de choisir aléatoirement une pièce dans la liste des lieux où l'on peut être téléporté. De plus, pour que la méthode getRandomRoom() fonctionne correctement, il faut importer la bibliothèque `java.util.Random`.

```
public class TransporterRoom extends Room{
	private Room[] aTargetRooms;

	public TransporterRoom(final String pDescription, final String pImage ){
		super(pDescription, pImage);
	}//TransporterRoom
}
```

```
TransporterRoom vTransporterRoom = new TransporterRoom("in the magic place",
"Images/teleporterRoom.jpg");
```

```
Room[] vTargetRooms = new Room[] {vBeach, vCoconuts, vCave, vVolcano, vCascade};
```

```
vTransporterRoom.setTargetRooms(vTargetRooms);
```

```
vTransporterRoom.setExit("down", vTransporterRoom);
```

```
vRiver.setExit("up", vTransporterRoom);
```

```
public void setTargetRooms(final Room[] pRooms){
	this.aTargetRooms = pRooms;
}
```

```
else if (this.aPlayer.getCurrentRoom().getDescription().equals("in the magic
place")){
	TransporterRoom
vTransporterRoom=(TransporterRoom)this.aPlayer.getCurrentRoom();
	Room vRandomDestination = vTransporterRoom.getRandomRoom();
	this.aPlayer.setCurrentRoom(vRandomDestination);
	this.aGui.println( this.aPlayer.getCurrentRoom().getLongDescription() );
	if ( this.aPlayer.getCurrentRoom().getImageName() != null )
		this.aGui.showImage( this.aPlayer.getCurrentRoom().getImageName() );
```

```
        return;
}
```

```
public Room getRandomRoom(){
        // random sur la taille du tableau
        Random vRandom = new Random();
        return aTargetRooms[vRandom.nextInt(this.aTargetRooms.length)];
}
```

```
import java.util.Random;
```

Exercice 7.46.1 :

Dans cet exercice, on souhaite ajouter une commande alea pour forcer la pièce de sortie de la Transported room. On ajoute le terme “alea” à la liste de commandes valides de la classe CommandWords. Ensuite, pour que la commande alea soit reconnue, on ajoute une condition dans la procédure interpretCommand() de la classe GameEngine. Pour savoir si on est en mode test ou pas, pour cela, on doit ajouter un attribut booléen aIsTest à la classe GameEngine. Cet attribut vaut **true** si on est en mode test sinon il vaut **false**. Il est initialisé à **false** dans le constructeur GameEngine(). Il faut modifier la méthode test() de la classe GameEngine pour que l’attribut aIsTest soit changé à **true**. On fait cette affectation après avoir ouvert le fichier et avant d’exécuter les commandes. On peut alors ajouter la méthode alea() à la classe GameEngine. Les instructions sont exécutées si le joueur est bien dans la Transporter room et si on est en mode test. Sinon il y a un message d’erreur. Le second mot après “alea” désigne le lieu dans lequel on souhaite être transporté en sortant de la Transporter room.. Lors de l’exercice 7.46, j’avais créé une liste des lieux où l’on pouvait être transporté, cette liste va être utilisée pour imposer la pièce dans laquelle on veut se téléporter lors des tests. On crée alors la méthode setForcedRoom(). Pour définir le lieu de destination. On utilise l’indice du tableau aTargetRooms pour récupérer l’objet Room qui sera retourné par la méthode getRandomRoom(). On crée également la méthode resetForcedRoom() qui supprime le lieu de destination. Ces deux méthodes sont utilisées par la méthode alea() de la classe GameEngine.

```
private boolean aIsTest;
```

```
public GameEngine(){
        this.createRooms();
        this.aParser = new Parser();
        this.aIsTest = false;
}//GameEngine
```

```
else if (vCommandWord.equals("alea"))
```

```
        alea(vCommand);
```

```
private void test (final Command pCommand){
	if(pCommand.hasSecondWord() == false){
		this.aGui.println("Need a file name");
		return;
	}

	String vFileName = pCommand.getSecondWord();
	vFileName = vFileName + ".txt";
	//lire le fichier
	Scanner vSC;
	InputStream
vIS=this.getClass().getClassLoader().getResourceAsStream(vFileName);
	if(vIS != null){
		this.aIsTest = true;
		vSC = new Scanner( vIS );
		while(vSC.hasNextLine()){
			String vLigne = vSC.nextLine();
			this.aGui.processCommand(vLigne);
		}
	}
	this.aIsTest = false;
}
```

```
private void alea(final Command pCommand){
	// ne fonctionne que dans la Transporter Room
	if (this.aPlayer.getCurrentRoom().getDescription().equals("in the magic
place") && this.aIsTest == true){
		TransporterRoom vTransporterRoom = (TransporterRoom)
this.aPlayer.getCurrentRoom();
		if(pCommand.hasSecondWord() == false){
				vTransporterRoom.resetForcedRoom();
		}
		else{
			String vRoom = pCommand.getSecondWord();
			vTransporterRoom.setForcedRoom(vRoom);
		}
	}
	else {
		this.aGui.println("you are not in the Transporter Room or in test
mode");
	}
	return;
}
```

```
public void setForcedRoom(final String pRoom){
	switch (pRoom){
		case "beach":
			this.aForcedRoom = this.aTargetRooms[0];
			break;
		case "coconuts":
			this.aForcedRoom = this.aTargetRooms[1];
			break;
		case "cave":
			this.aForcedRoom = this.aTargetRooms[2];
			break;
		case "volcano":
			this.aForcedRoom = this.aTargetRooms[3];
			break;
		case "cascade":
			this.aForcedRoom = this.aTargetRooms[4];
			break;
	}
}
```

```
public void resetForcedRoom(){
	this.aForcedRoom = null;
}
```

Exercices pour le scénario :

Exercice 1 :

l'objectif à l'issue de cet exercice est d'obtenir une clé lorsque l'on dépose un item dans un lieu. Pour mon jeu, un singe dans le lieu coconuts veut une banane qui se trouve dans la grotte en échange d'une clé. Pour obtenir cetteclé, il faut déposer la banane dans le lieu où est le singe. On commence par ajouter une méthode booléenne isInRoom() dans la classe Room qui renvoie true si le nom de l'item passé en paramètre est bien dans la pièce. Ensuite, on ajoute une méthode dropBananaInCoconuts() à la classe GameEngine elle permet de vérifier dans un premier temps si l'objet déposé est bien dans le lieu coconuts et si l'item déposé est bien la banane. Si ces deux conditions sont vérifiées, alors on enlève la banane qui vient d'être déposée (parce qu'elle est mangée par le singe) et on ajoute un nouvel item key dans le lieu coconuts. L'objet est déposé grâce à la méthode drop(). Il faut alors modifier la méthode drop() pour qu'elle appelle la méthode dropBananaInCoconuts() et vérifier si on est en train de déposer une banane pour le singe.

```
public boolean isInRoom(final String pNom){
```

```
        Set<String> vEnsembleCles = this.aItems.getKeys();
        for (String vCle : vEnsembleCles){
            if(pNom.equals(vCle)){
                return true;
            }
        }
        return false;
}
```

```
private void dropBananaInCoconuts(final Room pRoom){
        if(pRoom.getDescription().equals("at the coconut palms")){
                if (pRoom.isInRoom("banana")){
                        Item vItem = new Item("a key", 2);
                        this.aPlayer.getCurrentRoom().removeItem("banana");
                        this.aPlayer.getCurrentRoom().addItem("key", vItem);
                        this.aGui.println("monkeys have dropped a key!!!");
                }
        }
}
```

```
private void drop (final Command pCommand){
        if(pCommand.hasSecondWord() == false){
                this.aGui.println("Drop what?");
                return;
        }

        String vItemName = pCommand.getSecondWord();
        Item vItem = this.aPlayer.getItem(vItemName);
        if(vItem == null){
                this.aGui.println("I can't find any " + vItemName + "!");
                return;
        }
        this.aPlayer.getCurrentRoom().addItem(vItemName, vItem);
        this.aPlayer.removeItem(vItemName);
        int vPoids = this.aPlayer.getPoids() - vItem.getPoids();
        this.aPlayer.setPoids(vPoids);
        this.dropBananaInCoconuts(this.aPlayer.getCurrentRoom());
        this.aGui.println(this.aPlayer.getCurrentRoom().getLongDescription());
}
```

Exercice 2 :

Depuis le début du projet, je possède une salle au trésor. Cependant elle a toujours été accessible. Il faut donc fermer cette pièce et ne pouvoir l'ouvrir que si le joueur possède une clé

dans son inventaire (clé récupérable à l'Exercice 1). Il faut modifier la méthode goRoom(), pour vérifier si on cherche à entrer dans la salle du trésor et si on a bien la clé sur soi. Voici un extrait avec la condition à ajouter.

```
else if (vNextRoom.getDescription().equals("the room of the treasure")){
       String vItemList = this.aPlayer.getItemsList();
       String[] vList = vItemList.split(" ");
       for (int i = 0; i <vList.length; i++){
            if (vList[i].equals("key")){
                this.aPlayer.setPreviousRoom(this.aPlayer.getCurrentRoom());
                this.aPlayer.setCurrentRoom(vNextRoom);
                this.aGui.println(
this.aPlayer.getCurrentRoom().getLongDescription());
                if ( this.aPlayer.getCurrentRoom().getImageName() != null )
                    this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName()
);
                this.aPlayer.addDeplacement();
            }
       }
        this.aGui.println("you can't access this room. You need a key!");
}
```

Exercice 3 :

Maintenant, nous avons une condition pour accéder à la salle du trésor. Il n'y a par contre pas de méthode permettant de gagner, il faut gagner uniquement si on arrive dans la salle du trésor. On peut alors créer la méthode win() dans la classe GameEngine.. J'en ai profité pour simplifier un peu le code de la partie précédente.. Voici la nouvelle version qui permet de s'arrêter à la salle du trésor et de quitter le jeu car nous avons gagné.

```
private void win(){
       this.aGui.println("congratulations you find the treasure");
       this.endGame();
}
```

```
else if (vNextRoom.getDescription().equals("the room of the treasure")){
       String vItemList = this.aPlayer.getItemsList();
       String[] vList = vItemList.split(" ");
       for (int i = 0; i <vList.length; i++){
            if (vList[i].equals("key")){
                this.aPlayer.setCurrentRoom(vNextRoom);
                this.aGui.println(
this.aPlayer.getCurrentRoom().getLongDescription());
                if ( this.aPlayer.getCurrentRoom().getImageName() != null )
```

```
                    this.aGui.showImage(this.aPlayer.getCurrentRoom().getImageName()
);
                this.win();
            }
        }
        this.aGui.println("you can't access this room. You need a key!");
}
```

Exercice 4 :

Avec l'appel de la méthode endGame() il est encore possible d'utiliser les boutons alors que le jeu est terminé. Il faut donc empêcher les boutons de fonctionner. Pour ce faire, on crée une méthode buttonDisable() dans la classe UserInterface. On peut alors appeler cette méthode dans la procédure endGame() de la classe GameEngine.

```
public void buttonDisable() {
        JButton[] boutons = {aButtonNorth, aButtonSouth, aButtonEast, aButtonWest,
aButtonUp, aButtonDown, aButtonBack};
        for (int i = 0; i<boutons.length; i++) {
                    boutons[i].setEnabled(false);
        }
}
```

```
private void endGame(){
        this.aGui.println( "Thank you for playing.  Goodbye." );
        this.aGui.buttonDisable();
        this.aGui.enable( false );
}
```

III) Mode d'emploi (si nécessaire, instructions d'installation ou pour démarrer le jeu)

IV) Déclaration obligatoire anti-plagiat

Pour les images j'ai utilisé le site https://deepai.org/machine-learning-model/text2img
Pour les boutons de l'Exercice 7.18.8 (en particulier pour la méthode actionPerformed()): https://docs.oracle.com/javase/7/docs/api/javax/swing/JButton.html
Pour la méthode butonDisable() je me suis aidé d'openclassrooms et de stackoverflow.